Tema 2 - CSS y construcción de interfaces web



2.2 — Valores, unidades, funciones y modelo de cajas

Cómo interpreta CSS los valores, las unidades y las referencias de medida

En CSS no basta con indicar un número. También debemos indicar qué unidad utiliza ese número, porque cada unidad puede tomar una referencia diferente para calcular el tamaño.

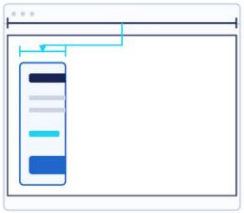
Vamos a modificar siempre la misma propiedad, `width`, sobre la misma tarjeta de curso. El número será `20` en los cuatro casos, pero el resultado será diferente porque cambia la unidad.

```
.tarjeta-curso
width: 20px;
```



`20px`
`px` expresa una longitud CSS absoluta. No depende del ancho del contenedor ni del viewport.

```
.tarjeta-curso
width: 20%;
```



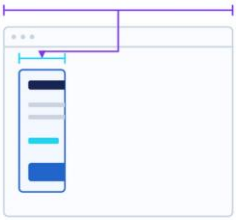
`20%`
En `width`, el porcentaje se calcula respecto al ancho disponible de su bloque contenedor.

```
.tarjeta-curso
width: 20rem;
```



`20rem`
`rem` se calcula respecto al tamaño de fuente del elemento raíz del documento.

```
.tarjeta-curso
width: 20vw;
```



`20vw`
`vw` se calcula respecto al ancho del viewport. `20vw` equivale al 20 % de ese ancho.

¿Qué ha cambiado?

La propiedad es siempre la misma: `width`. El número también es siempre el mismo: `20`. Lo que cambia es la **unidad** y, por tanto, la **referencia utilizada para calcular la medida**.

> **IDEA FUNDAMENTAL**
>
> No existe una unidad «mejor» para todo. La unidad adecuada depende de qué queremos medir y respecto a qué debe adaptarse.

Tipos de valores CSS: palabras clave, números, longitudes, porcentajes y funciones

Cada propiedad CSS admite determinados tipos de valores. No todos los valores pueden utilizarse con cualquier propiedad.



Palabras clave

`width: auto;` · `border-style: solid;`



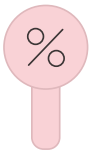
Números

`opacity: 0.8;` · `line-height: 1.5;`



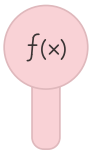
Longitudes

`padding: 1rem;` · `width: 24em;` · `border-width: 1px;`



Porcentajes

`width: 80%;` · `font-size: 120%;`



Funciones

`width: calc(100% - 2rem);` · `padding: clamp(1rem, 2vw, 2rem);`

✓ Válido

```
width: 300px;
```

✗ No válido

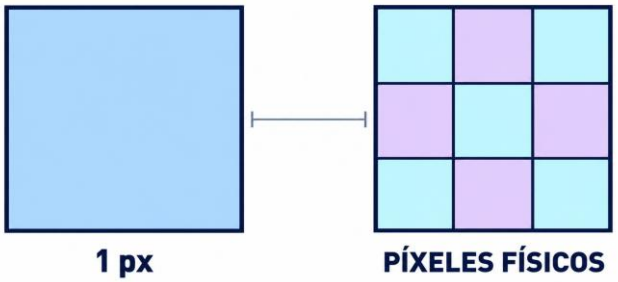
```
width: bold;
```

`bold` no representa un valor admitido por `width` y el navegador ignorará esa declaración.

El valor cero: cuando una longitud vale cero no suele ser necesario indicar unidad. `margin: 0;` en lugar de `margin: 0px;`

Unidades absolutas en CSS: píxeles CSS y medidas físicas

Las unidades absolutas mantienen una relación fija con la unidad de referencia definida por CSS. La unidad más utilizada en interfaces web es `px`.



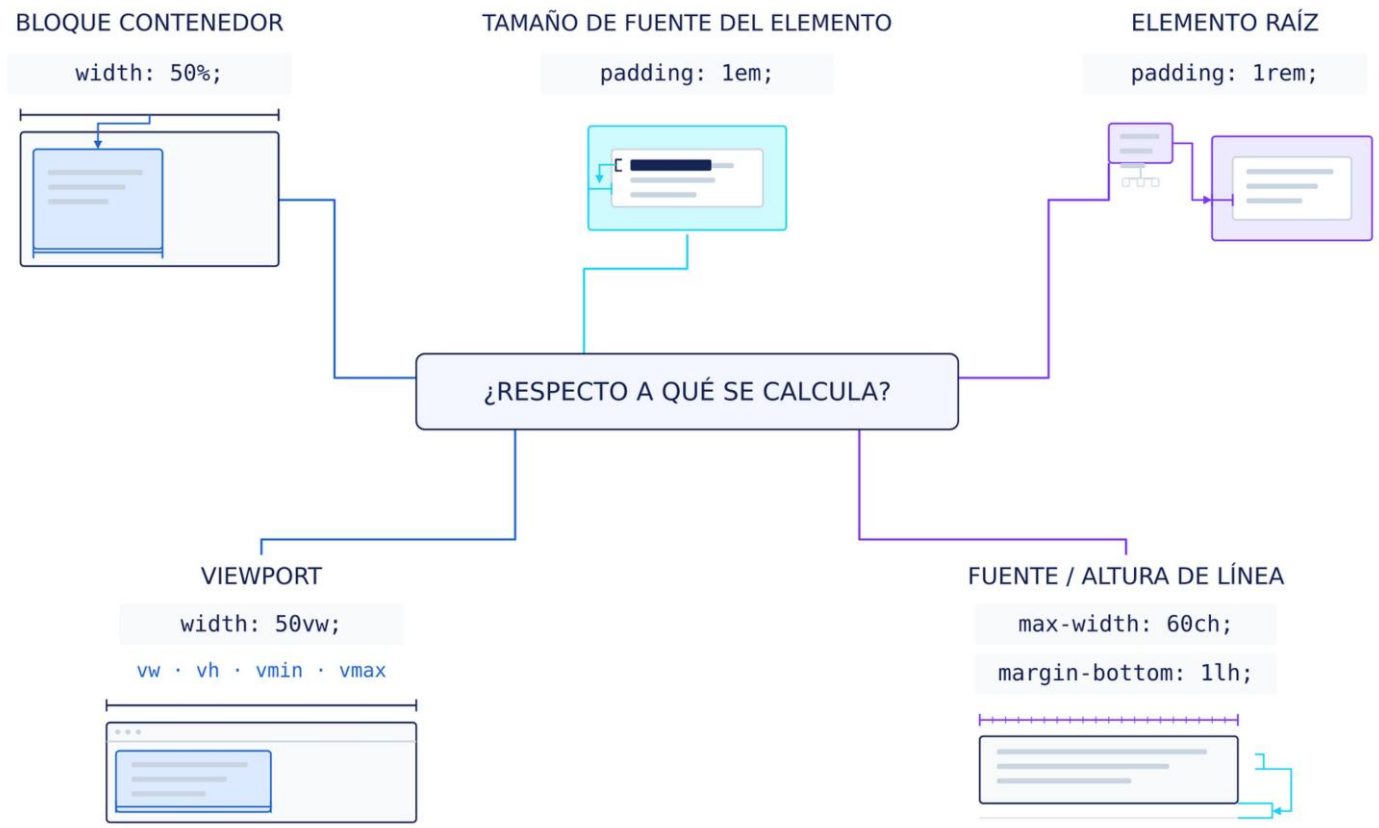
Un píxel CSS es una unidad de referencia del lenguaje. No debe confundirse necesariamente con un único píxel físico de una pantalla. En dispositivos de alta densidad, varios píxeles físicos pueden participar en la representación de un píxel CSS. Por ello, `width: 320px;` no significa que el elemento vaya a tener exactamente el mismo tamaño físico en centímetros en todos los dispositivos.

Unidad	Nombre	Equivalencia	Uso principal
`px`	Píxel CSS	Referencia base	Interfaces en pantalla
`in`	Pulgada	1in = 96px	Impresión
`cm`	Centímetro	1in = 2,54cm	Impresión
`pt`	Punto	1pt = 1/72in	Impresión
`pc`	Pica	1pc = 12pt	Impresión
`mm`	Milímetro	1 cm = 10 mm	Impresión

`border: 1px solid #cbd5e1;` puede resultar adecuado para medidas pequeñas y muy controladas. Fijar toda una interfaz exclusivamente mediante píxeles reduce su capacidad de adaptación.

Unidades relativas en CSS: medidas que dependen del contexto, la fuente o el viewport

Las unidades relativas calculan su tamaño tomando como referencia otro valor. Esto permite que una medida pueda cambiar cuando cambia su contexto. Las unidades principales son: `%` `em` `rem` `vw` `vh` `vmin` `vmax` `ch` `lh`. No todas toman como referencia lo mismo.



> Una unidad relativa no significa simplemente «una medida que cambia». Significa una medida calculada respecto a una referencia determinada.

La unidad em: cálculo según el tamaño de fuente del padre o del propio elemento

La unidad `em` está relacionada con el tamaño de fuente. Sin embargo, su referencia depende de la propiedad en la que se utiliza.

em en font-size

Cuando `em` se utiliza para definir `font-size`, toma como referencia el tamaño de fuente calculado del **elemento padre**.

```
.tarjeta { font-size: 20px; }
.tarjeta h2 { font-size: 1.5em; }
```

20px × 1.5 = **30px**

Referencia: tamaño de fuente del padre.

em en width / padding

Cuando `em` se utiliza en propiedades como `width`, `padding`, `margin` o `gap`, se calcula respecto al tamaño de fuente calculado del **propio elemento**.

```
.tarjeta { font-size: 16px; width: 20em; }
/* 20 × 16px = 320px */

.tarjeta { font-size: 20px; width: 20em; }
/* 20 × 20px = 400px */

.boton { font-size: 16px;
         padding: 0.75em 1.5em; }
```

Distinción fundamental

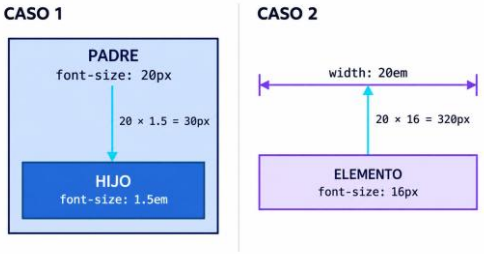
`font-size: 1.5em` → tamaño de fuente del **padre**

`width: 20em` → tamaño de fuente del **propio elemento**

`padding: 1em` → tamaño de fuente del **propio elemento**

Efecto acumulativo en font-size

```
.contenedor { font-size: 1.2em; }
.contenedor p { font-size: 1.2em; }
```



Si el tamaño heredado inicial fuese `16px`:

`contenedor → 16 × 1.2 = 19.2px`

`p → 19.2 × 1.2 = 23.04px`

Por eso los `em` utilizados en `font-size` pueden acumularse cuando existen varios niveles anidados.

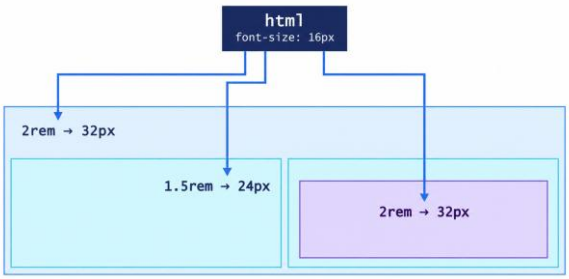
La unidad rem: cálculo respecto al tamaño de fuente del elemento raíz

`rem` significa *root em*. Su referencia es el tamaño de fuente calculado del elemento raíz: `html`. A diferencia de `em`, el resultado no depende del tamaño de fuente de elementos intermedios.

```
html { font-size: 16px; }

h1 { font-size: 2rem; }        /* 2rem = 32px */

.tarjeta {
  padding: 1.5rem;
  margin-bottom: 2rem;
}
```



⚠ No debemos suponer que 1rem siempre son 16px. Muchos navegadores utilizan inicialmente un tamaño equivalente a 16px, pero el usuario puede modificar sus preferencias. No conviene forzar `html { font-size: 16px; }` únicamente para convertir mentalmente rem en píxeles.

¿Dónde puede resultar útil `rem`?



> rem utiliza una referencia común: el tamaño de fuente del elemento raíz.

Porcentajes en CSS: una referencia que depende de la propiedad utilizada

El símbolo `%` expresa un valor relativo. Sin embargo, no existe una única referencia válida para todos los porcentajes de CSS. Cada propiedad define respecto a qué se calcula su porcentaje.

1 Ejemplo — width

```
.contenedor { width: 800px; }
.tarjeta    { width: 50%;   }
```

Si la referencia es 800px: **50% → 400px**

El porcentaje de `width` toma como referencia el ancho del bloque contenedor.

2 Ejemplo — font-size

```
p { font-size: 120%; }
```

Aquí el porcentaje se calcula respecto al tamaño de fuente heredado. La misma notación, una referencia diferente.

`width: 50%`

800px contenedor

↓

400px resultado

`font-size: 120%`

Si el tamaño de fuente heredado es `16px`:

`16px × 120% = 19.2px`

Conclusión

50% ≠ siempre «la mitad del padre»

¿Respecto a qué define esta propiedad su porcentaje?

Unidades del viewport: vw, vh, vmin, vmax y unidades modernas svh, lvh y dvh

El viewport es el área utilizada por el navegador para mostrar la página.

`vw`

100vw = 100% del ancho del viewport.

```
.hero { width: 80vw; }
```

`vh`

100vh = 100% de la altura del viewport.

```
.hero { min-height: 60vh; }
```

`vmin`

Utiliza la dimensión menor.

```
.decoracion { width: 10vmin; }
```

`vmax`

Utiliza la dimensión mayor.

```
.decoracion { width: 10vmax; }
```

Viewport en dispositivos móviles

Las barras del navegador pueden modificar el área visible. Aquí nos interesa comprender la referencia de cada unidad.

`svh`

small viewport height

Utiliza como referencia la altura del **viewport pequeño**, normalmente asociada al espacio disponible cuando las barras dinámicas del navegador están visibles.

`lvh`

large viewport height

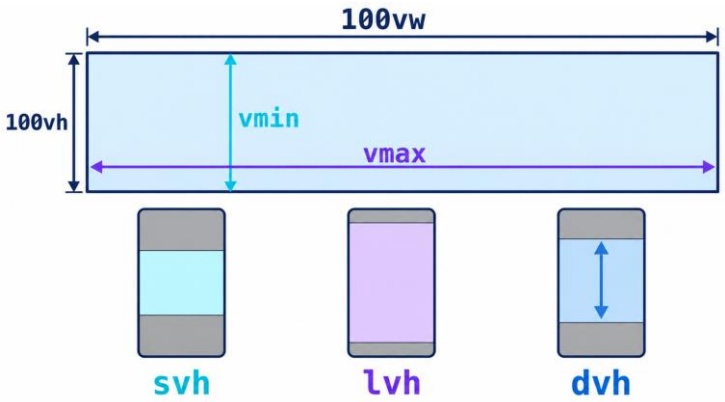
Utiliza como referencia la altura del **viewport grande**, asociada al mayor espacio disponible cuando esas barras están retraídas.

`dvh`

dynamic viewport height

Su referencia se actualiza dinámicamente cuando cambia el espacio visible del viewport.

```
.pantalla { min-height: 100dvh; }
```



Cómo elegir unidades CSS según la referencia y el comportamiento que necesita cada medida

No existe una regla que obligue a utilizar siempre unidades relativas. Tampoco utilizar `px` es incorrecto por sí mismo. La elección depende del comportamiento que necesitamos.

Necesidad	Unidad recomendada
Borde fino y controlado	`px`
Escala global de texto y espacios	`rem`
Medida ligada al texto de un componente	`em`
Dimensión relativa al contenedor	`%`
Dimensión relativa al viewport	`vw`, `vh`, `vmin`, `vmax`
Longitud aproximada de una línea de texto	`ch`
Espaciado relacionado con la altura de línea	`lh`
Medidas destinadas a impresión	`cm`, `mm`, `pt`, `in`

Un mismo componente puede combinar unidades

```
.tarjeta {
  width: 90%;          /* adapta el ancho al espacio disponible */
  max-width: 32rem;    /* establece un límite                   */
  padding: 1.5rem;     /* mantiene un espaciado coherente       */
  border: 1px solid #cbd5e1;  /* crea un borde fino             */
}
```

`ch`

```
.texto { max-width: 65ch; }
```

Se basa en una medida relacionada con el ancho del carácter «0» de la fuente.

`lh`

```
p { margin-bottom: 1lh; }
```

1lh corresponde a la altura de línea calculada del propio elemento.

La función calc(): combinar porcentajes y longitudes mediante cálculos CSS

`calc()` permite realizar cálculos dentro de un valor CSS. Una de sus principales ventajas es combinar medidas con referencias distintas.

1

100%

Ancho disponible del contenedor

2

– 2rem

Resta una medida expresada en `rem`.

3

Resultado

El navegador evalúa la expresión en tiempo real

```
.contenido { width: calc(100% - 2rem); }

.panel { min-height: calc(100vh - 4rem); }
```

Suma

```
width: calc(50% + 2rem);
```

Resta

```
height: calc(100vh - 5rem);
```

Sintaxis correcta

```
calc(100% - 2rem)
```

Los operadores + y – deben quedar claramente separados.

Sintaxis incorrecta

```
calc(100%-2rem)
```

Sin espacios alrededor del operador.

El navegador puede volver a evaluar la expresión cuando cambia el contexto.

> `calc()` permite expresar relaciones entre medidas.

Las funciones min() y max(): seleccionar dinámicamente el menor o el mayor valor

CSS puede comparar diferentes medidas mediante `min()` y `max()`.

`min()`

Selecciona el **menor** de los valores indicados.

```
.contenedor {
  width: min(90%, 70rem);
}
```

Si 90% es menor → se utiliza 90%.
Si 90% supera 70rem → se utilizan 70rem.

Conceptualmente: «ocupa gran parte del espacio disponible, pero no superes 70rem».

```
width: min(100%, 40rem);
```

`max()`

Selecciona el **mayor** de los valores.

```
.cabecera {
  padding-top: max(1rem, 3vh);
}
```

Puede combinar referencias diferentes.

```
padding: max(1rem, 2vw);
```

El resultado siempre será al menos `1rem`, pero puede crecer si `2vw` resulta mayor.

`min()`	`max()`
→ menor resultado de los valores comparados	→ mayor resultado de los valores comparados

La función clamp(): limitar un valor flexible entre un mínimo y un máximo

`clamp()` permite expresar un valor flexible dentro de unos límites.

Sintaxis

```
clamp(MÍNIMO, VALOR PREFERIDO, MÁXIMO)
```

Mínimo	Preferido	Máximo
`1rem` El valor nunca será menor	`2vw` El navegador intenta utilizar este valor	`2rem` El valor nunca será mayor

```
.tarjeta { padding: clamp(1rem, 2vw, 2rem); }
```

El navegador intenta utilizar `2vw`, pero el resultado nunca será menor que `1rem` ni mayor que `2rem`.

```
.titulo { font-size: clamp(1.75rem, 4vw, 3rem); }
```

Permite expresar simultáneamente un límite inferior, un valor flexible vinculado al viewport y un límite superior para el tamaño de fuente.



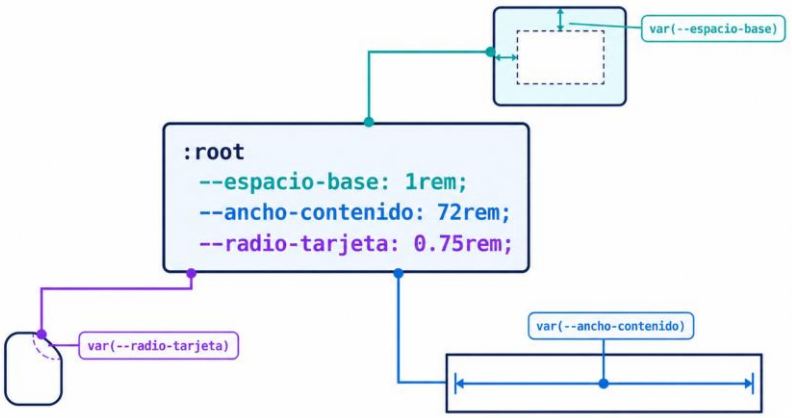
Propiedades personalizadas de CSS: declarar variables reutilizables mediante --nombre

CSS permite crear propiedades cuyo valor podemos reutilizar. Se denominan **propiedades personalizadas**, aunque también es habitual llamarlas **variables CSS**. Sus nombres comienzan con dos guiones: `--espacio-base`, `--ancho-contenido`, `--radio-tarjeta`. Forman parte del propio CSS y son interpretadas por el navegador.

```
:root {
  --espacio-base:    1rem;
  --espacio-grande:  2rem;
  --ancho-contenido: 72rem;
}
```

¿Por qué `:root`?
`:root` selecciona el elemento raíz del documento, que en HTML corresponde a `<html>`. Declarar aquí propiedades personalizadas permite que estén disponibles, mediante herencia, en prácticamente todo el documento.

```
.tarjeta    { padding:    var(--espacio-base);    }
.contenedor { max-width:  var(--ancho-contenido); }
```



Si modificamos `--espacio-base: 1.25rem;`, las declaraciones que utilizan esa propiedad obtienen el nuevo valor. Los nombres distinguen mayúsculas y minúsculas: `--espacio` y `--Espacio` son diferentes.

La función var(): reutilizar variables CSS, controlar su ámbito y definir valores de respaldo

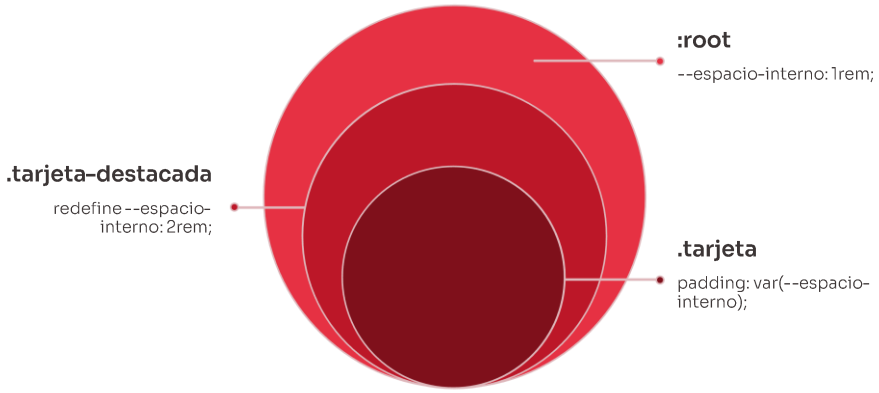
`var()` permite utilizar el valor de una propiedad personalizada.

```
:root { --espacio-interno: 1rem; }

.tarjeta { padding: var(--espacio-interno); }
```

Ámbito y herencia

Una propiedad personalizada puede declararse dentro de un elemento concreto. Las propiedades personalizadas se heredan normalmente hacia los descendientes.




```
<section class="tarjeta-destacada">
  <article class="tarjeta">
    Curso destacado
  </article>
</section>
```

```
:root { --espacio-interno: 1rem;  }
.tarjeta-destacada { --espacio-interno: 2rem; }
.tarjeta { padding: var(--espacio-interno); }
```

`.tarjeta` es descendiente de `.tarjeta-destacada`, por lo que hereda de ella `--espacio-interno: 2rem`. En otra zona del documento donde no exista esa redefinición, utilizaría el valor general de `1rem` definido en `:root`.

Valor de respaldo

Variable válida

Se utiliza el valor de la propiedad personalizada.

```
padding: var(--espacio-interno, 1rem);
```

Variable no disponible o sin valor válido

Se utiliza el valor de respaldo: `1rem`.

```
max-width: var(--ancho-contenido, 70rem);
```

Las propiedades width, height y auto: definir dimensiones sin impedir la adaptación al contenido

`width` y `height` permiten controlar las dimensiones de una caja.

1 Definir el ancho con width

`width` permite indicar la anchura que debe utilizar un elemento.

Puede expresarse mediante una medida determinada, o mediante una relativa:

```
.tarjeta { width: 30rem; }
.tarjeta { width: 80%;   }
```

En el segundo caso, el ancho se calcula respecto al espacio de referencia del contenedor. Por tanto, `width` no implica necesariamente una dimensión fija.

2 height fijo — riesgo de desbordamiento

```
.banner { height: 20rem; }
```

Si el contenido necesita más espacio puede producirse desbordamiento.

3 height: auto — adaptación al contenido

```
.tarjeta { height: auto; }
```

La altura puede adaptarse al contenido. En muchos elementos de bloque, el ancho automático permite utilizar el espacio disponible.

Altura fija

El contenido puede desbordar si crece.

Altura adaptada al contenido

Cuando el contenido puede variar, suele ser preferible permitir que la altura se adapte y establecer límites solo cuando sean necesarios.

Límites de tamaño con min-width, max-width, min-height, max-height y dimensionamiento intrínseco

CSS permite limitar dimensiones sin obligar a utilizar un único tamaño fijo.

Propiedad	Ejemplo	Efecto
`max-width`	`.contenedor { width: 90%; max-width: 72rem; }`	Establece un límite superior de anchura.
`min-width`	`.panel { min-width: 18rem; }`	Establece un límite inferior de anchura.
`min-height`	`.hero { min-height: 20rem; }`	Establece un límite inferior de altura.
`max-height`	`.panel { max-height: 30rem; }`	Establece un límite superior de altura.

`height: 20rem;` establece una altura concreta. `min-height: 20rem;` establece un límite inferior: la caja puede ser más alta si el contenido lo requiere.

¿Qué significa que el contenido tenga un tamaño intrínseco?. En algunos casos podemos dejar que CSS determine la anchura teniendo en cuenta cuánto espacio necesita realmente el propio contenido.

Dimensionamiento intrínseco

`min-content`

```
width: min-content;
```

Intenta reducir la caja hasta el menor ancho que permite el contenido. En un texto, se producen saltos de línea en todas las oportunidades posibles.

`max-content`

```
width: max-content;
```

Intenta proporcionar al contenido todo el ancho que necesita sin provocar saltos de línea simplemente por falta de espacio.

Si el contenido necesita más altura que `max-height`, puede producirse desbordamiento; estudiaremos cómo gestionarlo mediante `overflow`.

`fit-content`

```
width: fit-content;
```

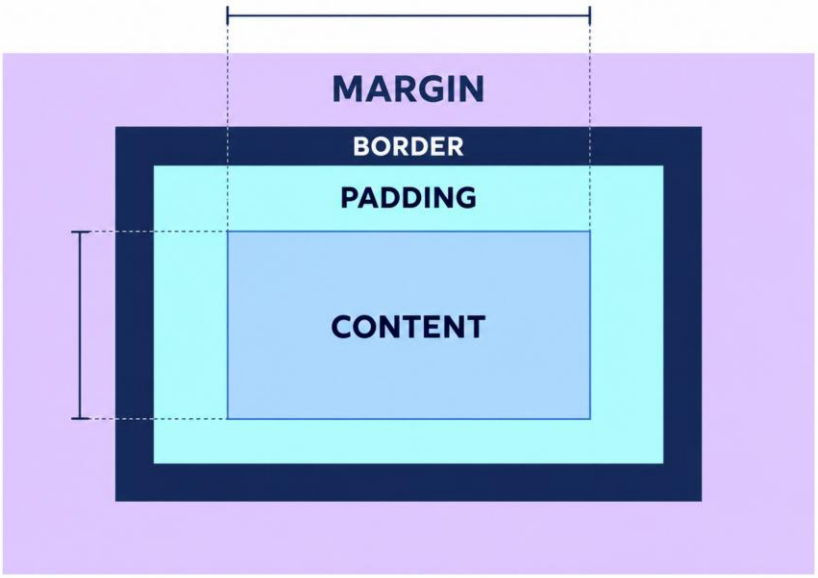
Ajusta la caja al tamaño de su contenido, pero evita que crezca por encima del espacio disponible.

```
.etiqueta { width: fit-content; }
```

Si el texto de la etiqueta es corto, la caja puede ocupar únicamente el espacio que necesita en lugar de extenderse por todo el contenedor.

Modelo de cajas CSS: contenido, padding, border y margin de cada elemento

CSS representa los elementos mediante cajas. Una caja está formada por cuatro zonas. El tamaño ocupado no depende únicamente de `width` y `height`. También intervienen `padding`, `border` y `margin`. La forma en que `width` y `height` se relacionan con estas zonas depende de `box-sizing`.



Content

Espacio donde se representa el contenido.

Padding

Espacio entre el contenido y el borde.

Border

Rodea contenido y padding.

Margin

Separación exterior respecto a otras cajas.

Cómo calcular el tamaño real de una caja con width, padding, border y margin

```
.tarjeta {
  width:         300px;
  padding-left:   20px;
  padding-right:  20px;
  border-left:     2px solid;
  border-right:    2px solid;
}
```

Con `box-sizing: content-box;` los 300px corresponden únicamente al contenido.



300 + 20 + 20 + 2 + 2 = 344 px

300px

Width declarado

Solo el área de contenido.

344px

Caja hasta el borde

Incluye padding y border.

Si además se añade `margin-left: 16px; margin-right: 16px;`, el espacio necesario para colocar la caja será mayor. La caja hasta el borde sigue midiendo `344px`, porque `margin` está fuera de ella. Para colocar el elemento necesitamos: `16 + 344 + 16 = 376px`

Por tanto:

ancho hasta el borde → 344px
espacio horizontal incluyendo márgenes → 376px

> `width: 300px;` no significa necesariamente que todo el elemento ocupe 300px.

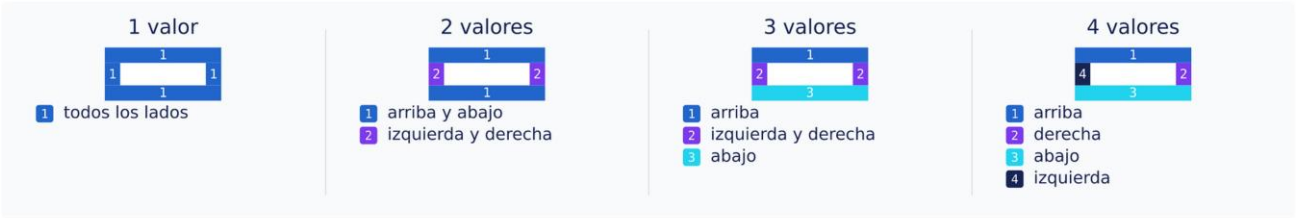
La propiedad padding: controlar el espacio interior y utilizar su sintaxis abreviada

`padding` crea espacio entre el contenido y el borde.

```
padding-top:    1rem;
padding-right:  1.5rem;
padding-bottom: 1rem;
padding-left:   1.5rem;
```

```
.tarjeta {
  padding: 1rem 2rem;
}
```





1 valor: todos los lados

2 valores: superior e inferior / izquierda y derecha

3 valores: superior / izquierda y derecha / inferior

4 valores: superior / derecha / inferior /izquierda

El recorrido se realiza en sentido horario desde arriba. El `padding` forma parte de la caja. Su efecto en el tamaño depende de `box-sizing`.

Las propiedades border y border-radius: definir el límite, el estilo y las esquinas de una caja

El borde rodea contenido y padding.

Tres componentes

```
border-width: 1px;
border-style: solid;
border-color: #cbd5e1;
```

Forma abreviada

```
border: 1px solid #cbd5e1;
```

Orden: grosor | estilo | color.

Bordes independientes

```
border-top:    4px solid #2266cc;
border-bottom: 1px solid #cbd5e1;
```

Valores de border-style

`solid`

Línea continua.

`dashed`

Línea discontinua.

`dotted`

Línea de puntos.

`double`

Línea doble.

`none`

Sin borde visible.

border-radius

```
border-radius: 0.75rem;
```

Un valor aplica el mismo radio a las cuatro esquinas. `border-radius` modifica la forma visual pero no añade espacio adicional.

```
border-radius: 1rem 2rem;
```

Dos valores: superior izquierda e inferior derecha / superior derecha e inferior izquierda.

```
border-radius: 1rem 2rem 3rem 4rem;
```

Cuatro valores: superior izquierda → superior derecha → inferior derecha → inferior izquierda.

La propiedad margin y el valor auto: controlar la separación exterior y centrar bloques

`margin` crea espacio exterior alrededor del borde. Las reglas de uno, dos, tres y cuatro valores siguen el mismo orden que `padding`.

```
margin-top:    …; margin-right:  …; margin-bottom: …; margin-left:   …;

margin: 1rem 2rem;  /* forma abreviada */
```

El margen está fuera del borde. El fondo no se pinta sobre el margen.

El valor auto

```
.contenedor {
  width:      90%;
  max-width:  72rem;
  margin:     0 auto;
}
```




```
.contenedor {
  width: 90%;
  max-width: 72rem;
  margin: 0 auto;
}
```

`width: 90%` determina el ancho porque todavía no se alcanza el límite establecido por `max-width`.

Cuando el 90 % sería mayor que `72rem`, `max-width` limita el crecimiento. El espacio horizontal sobrante se reparte entre los dos márgenes automáticos.

Colapso de márgenes verticales: por qué dos márgenes consecutivos no siempre se suman

Determinados márgenes verticales de cajas de bloque en flujo normal pueden colapsar.

```
.primero { margin-bottom: 32px; }
.segundo { margin-top:    20px; }
```

Expectativa

Caja 1

↓ margin-bottom: 32px

↑ margin-top: 20px

Caja 2

~~32px + 20px = 52px~~

Incorrecto

Resultado real

Caja 1

↓ **32px** (prevalece el mayor)

Caja 2

Sin separación adicional

Espacio resultante: **32px**

Correcto

Márgenes horizontales

`margin-left` y `margin-right` no colapsan de esta forma.

Relaciones padre-hijo

En determinadas condiciones puede existir también colapso entre el margen superior del contenedor y el primer hijo, o entre el margen inferior y el último hijo.

Flexbox / Grid

Los márgenes de elementos Flexbox o Grid no presentan este mismo colapso entre elementos.

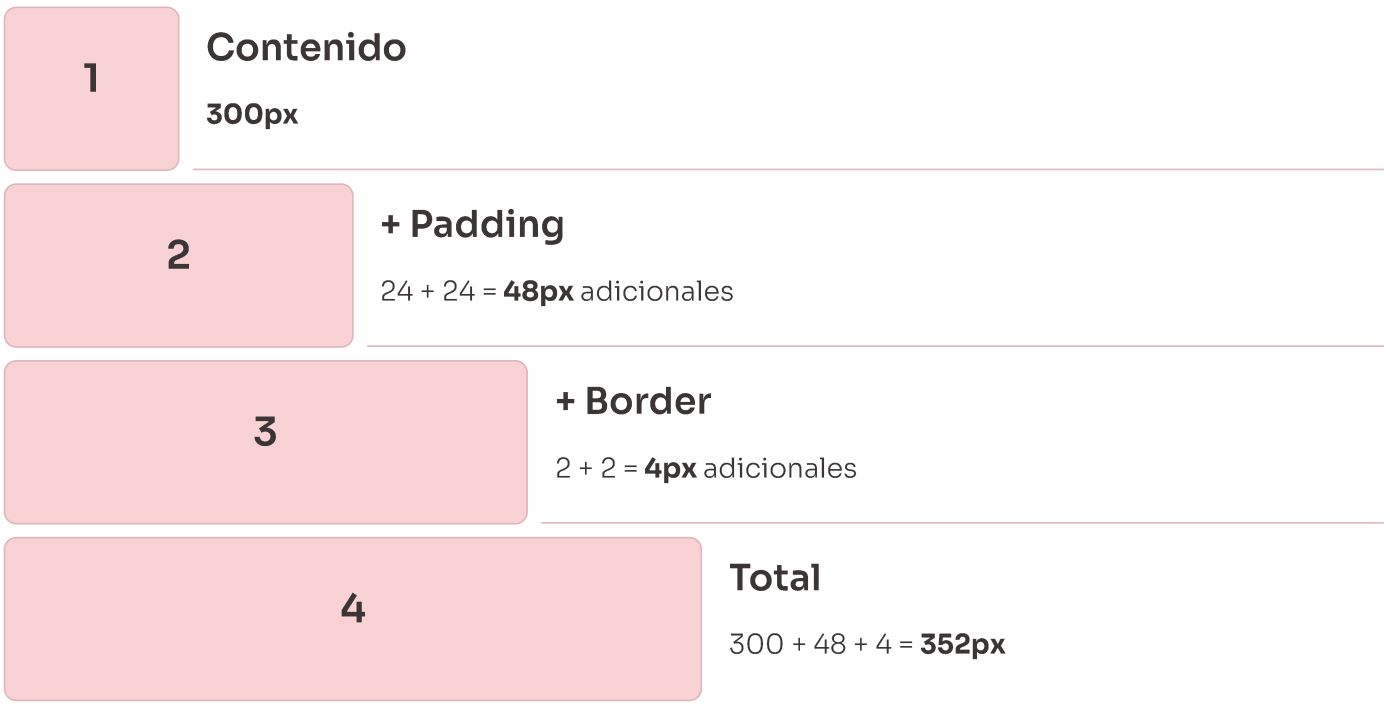
> Dos márgenes verticales no siempre se suman.

box-sizing: content-box: cuando width y height solo miden el contenido

`box-sizing` determina qué zonas de la caja están incluidas en `width` y `height`. Su valor inicial es `content-box`.

```
.tarjeta {
  box-sizing: content-box;
  width:      300px;
  padding:     24px;
  border:       2px solid #cbd5e1;
}
```

Con `content-box`, los 300px se aplican únicamente al contenido. Padding y border se añaden hacia el exterior.



300px

Width declarado

Solo el área de contenido.

352px

Caja hasta el borde

Añadir padding o border incrementa el tamaño exterior.

box-sizing: border-box: incluir padding y border dentro de las dimensiones declaradas

Con `box-sizing: border-box;`, `width` y `height` incluyen `content` + `padding` + `border`.

```
.tarjeta {
  box-sizing: border-box;
  width:      300px;
  padding:     24px;
  border:       2px solid #cbd5e1;
}
```

El ancho total hasta el borde será **300px**. Dentro de ese ancho: padding = 48px, border = 4px, contenido = 300 − 48 − 4 = **248px**.

content-box

`width: 300px` → solo contenido

Caja hasta el borde: **352px**

300px	+52px
contenido	padding + border

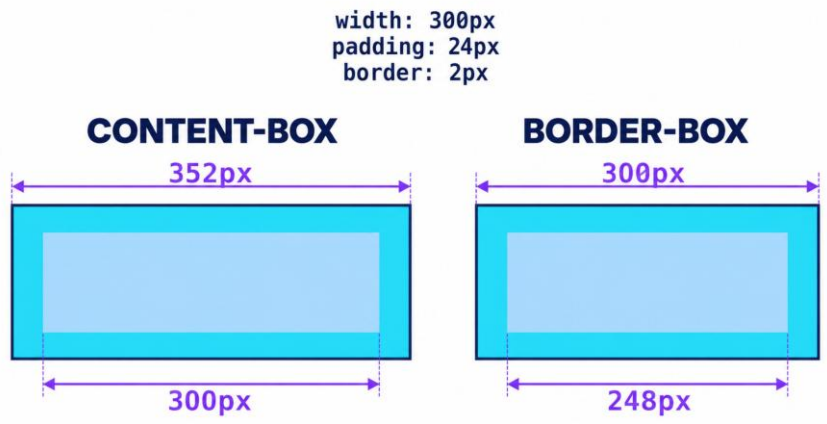
border-box

`width: 300px` → incluye todo

Caja hasta el borde: **300px**

248px	52px
contenido	padding + border

> `border-box` facilita relacionar la dimensión declarada con el tamaño exterior hasta el borde. El margen permanece siempre fuera.

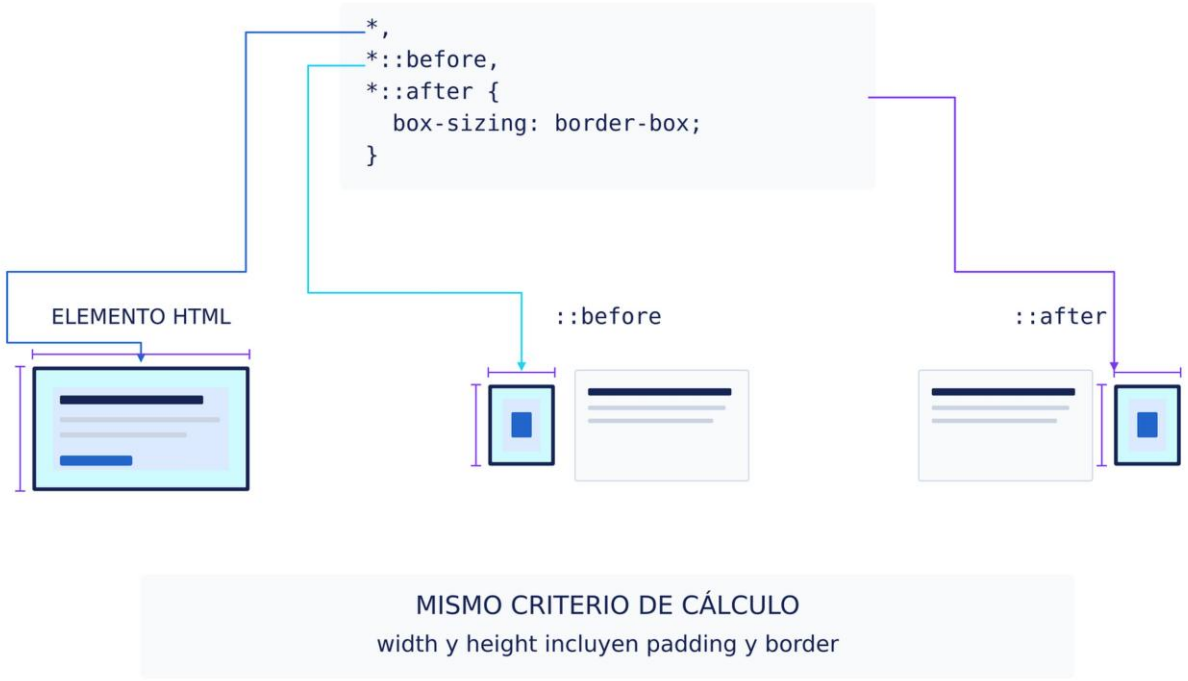


Aplicar box-sizing: border-box de forma global con *, ::before y ::after

Podemos establecer un criterio común para todas las cajas.

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

La regla se aplica a todos los elementos y a sus pseudoelementos `::before` y `::after`.




```
.tarjeta {
  width:   100%;
  padding: 1.5rem;
  border:  1px solid #cbd5e1;
}
```

Con `box-sizing: border-box;`, `width: 100%;` incluye `padding` y `border`. Por tanto, añadir estos valores no incrementa el ancho exterior más allá del 100% declarado.

¿Por qué incluir ::before y ::after?

Los pseudoelementos pueden generar cajas. También pueden utilizar `width`, `height`, `padding` y `border`. Aplicar `border-box` mantiene un mismo criterio de cálculo.

> Un modelo de cajas coherente reduce cálculos inesperados.

Caso completo: dimensionar una tarjeta con unidades, límites, variables CSS y box-sizing

Queremos construir una tarjeta de curso que utilice el espacio disponible, no crezca excesivamente, tenga espaciado interior flexible, permanezca centrada y mantenga un tamaño predecible.

1. Variables

```
:root {
  --ancho-tarjeta:  34rem;
  --espacio-tarjeta: 1.5rem;
  --radio-tarjeta:  0.75rem;
}
```

2. Modelo de cajas

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

3. Tarjeta

```
.tarjeta-curso {
  width:         100%;
  max-width:     var(--ancho-tarjeta);
  padding:       clamp(1rem, 2vw, var(--espacio-tarjeta));
  margin:        0 auto;
  border:        1px solid #cbd5e1;
  border-radius: var(--radio-tarjeta);
}
```

`width: 100%`
Utiliza el ancho disponible.

`max-width`
Limita su crecimiento.

`padding: clamp(...)`
Adapta el espacio interior.

`margin: 0 auto`
Centra la caja.

`border`
Define el límite.

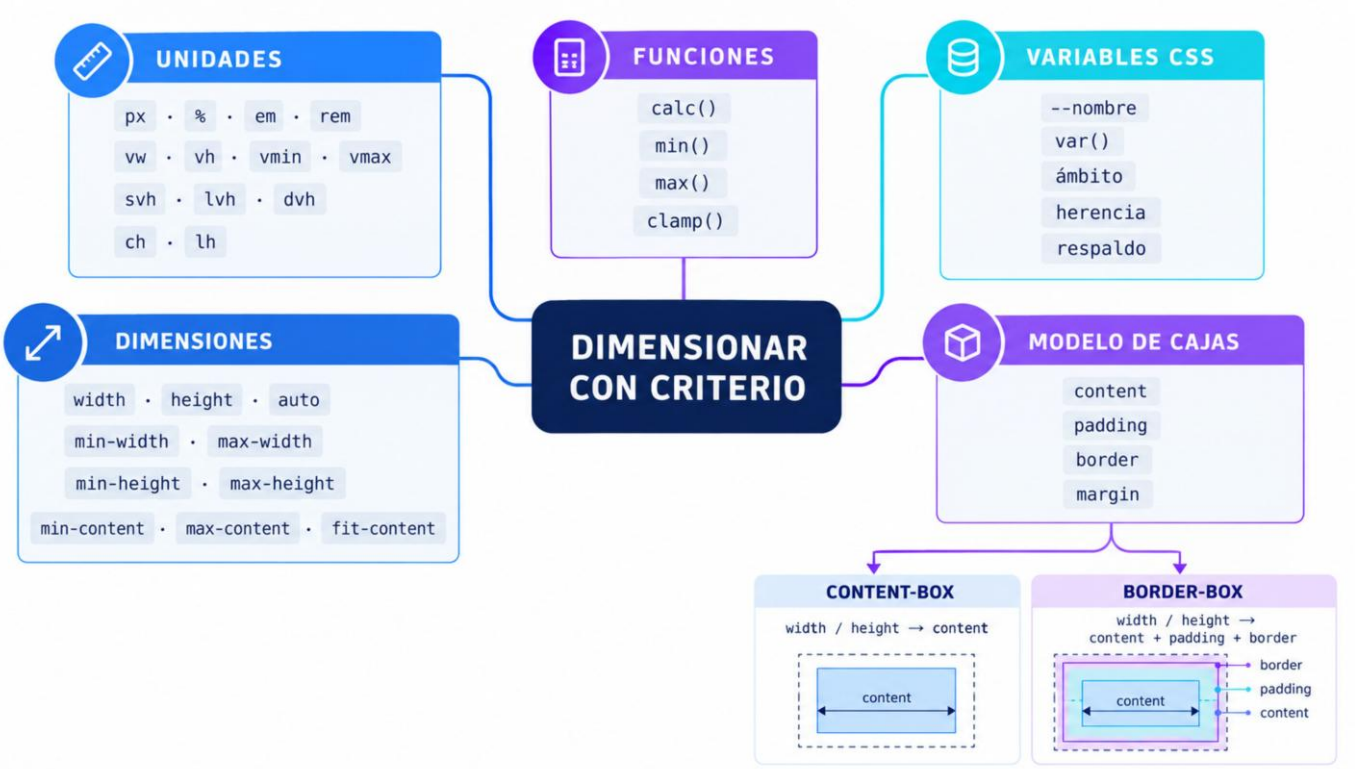
`border-radius`
Reutiliza una variable.

`box-sizing: border-box`
Mantiene padding y border dentro del ancho.

> El componente combina unidades relativas, límites, funciones, variables CSS y modelo de cajas.

Síntesis de valores, unidades, funciones y modelo de cajas en CSS

Una medida CSS no se comprende únicamente por el número. La unidad define su referencia.



> Construir una interfaz flexible consiste en decidir: qué debe permanecer estable; qué debe depender del contexto; qué referencia necesita cada medida; qué límites debe respetar cada componente.